

Milestone 1 Project Proposal

Course: CIS-5980

Track: AI Engineering

Team: Will Liu, Guadalupe Cantera, and Bryan Yang

Repository: <https://github.com/BryanDYang/ai-capstone>

1. Problem statement

Weekly research meetings produce commitments, professor suggestions, decisions, deadlines, and follow-up questions that are difficult to maintain across weeks. A researcher may report progress on something assigned several meetings earlier, refer to an old task using shorthand, change a deadline, or decide not to pursue a previous suggestion. Traditional meeting summaries can capture what happened in an individual meeting, but users still have to connect those updates to earlier conversations to determine what is still open, what changed, who is responsible, and why a decision was made.

This matters because losing that context across recurring meetings can lead to duplicated work, forgotten commitments, unclear ownership, missed follow-ups, and decisions whose original reasoning becomes difficult to recover. The existing CCB meeting pipeline already handles recording, transcription, speaker identification, transcript cleanup, and meeting summaries, so our project focuses specifically on the follow-through problem rather than rebuilding that existing functionality: maintaining and updating commitments, decisions, and their supporting context as they evolve across meetings.

Our central AI-engineering question is: Can a persistent, evidence-backed approach maintain commitments across a sequence of meetings more reliably than treating each meeting independently, while reducing manual reconciliation and avoiding unsupported task updates?

2. Target users and context

Our primary users are PhD advisors and the researchers or students who meet with them regularly. The intended setting is recurring research-group meetings, such as weekly lab or advisor meetings, where participants discuss ongoing work, make decisions, assign or accept commitments, set deadlines, and follow up on work discussed in previous meetings. Both advisors and researchers benefit from having a clearer record of what was decided, who committed to what, what remains unresolved, and how those items have changed over time.

For development and evaluation, our own consenting project team and study groups will serve as a secondary usage setting. We will use these groups to purpose-record controlled multi-meeting scenarios that reflect the types of commitments, decisions, suggestions, updates, and follow-ups expected in the primary research-group setting, as described in the Data section.

3. Value proposition

The system turns a sequence of disconnected meeting artifacts into an evidence-backed project memory. Instead of requiring users to manually compare summaries from different weeks, it maintains the current state of commitments, decisions, deadlines, and suggestions while preserving where each item came from and how it changed. This reduces the manual work required to reconstruct what happened across previous meetings while keeping the underlying evidence available for verification.

Success from the user's perspective means being able to open the system before or after a meeting and quickly understand what is still open, what was completed, blocked, dropped, or changed, who owns each commitment, which professor suggestions were actually accepted, what decisions were made and why, and what meeting evidence supports that information. The user should spend less time searching through previous transcripts or summaries and more time acting on a clear, trustworthy view of where the project currently stands.

4. End deliverable

By the end of the capstone, we aim to deliver a reproducible, local application consisting of a Python background service and a native iOS user interface. The system will import timestamped meeting artifacts, associate them with the appropriate project and calendar context, extract structured commitments, decisions, deadlines, and suggestions, and maintain that information across recurring meetings. As later meetings are processed, the system will reconcile new statements with existing records so that commitments can be updated rather than recreated or lost.

Through the native iOS interface, users will be able to review the current state of a project, inspect commitments and their supporting evidence, review ambiguous updates before they are applied, make corrections, delete records, view meeting and pre-meeting summaries, and ask historical questions that return answers with source citations. The final deliverable will also include a reproducible evaluation harness, labeled multi-meeting sequences, baseline comparisons, measured results, setup documentation, and a live demonstration of the system working across a three-meeting sequence.

5. Success metrics

Offline technical metrics:

Our main technical goal is to measure how accurately the system can identify commitments and keep them updated across multiple meetings. For extracted commitments, we will measure precision and recall against human-labeled examples, with an initial target of at least 90% precision and 80% recall. For cross-meeting updates, we will measure whether the system connects a later statement to the correct existing commitment and makes the correct status, owner, or deadline change. Our initial target is at least 95% precision for updates that the system applies automatically.

We will also compare our system against a baseline that processes each meeting independently without access to information from previous meetings. We want to see whether maintaining persistent meeting history results in a more accurate current task list, with fewer duplicate tasks, outdated items, and unsupported changes. We will pay particular attention to false completions, where the system incorrectly marks something as done without enough evidence from the meeting. These targets are starting points and may be adjusted based on what we learn during development and validation.

Product and user experience goals:

From the user's perspective, success means being able to quickly understand where a project currently stands without having to search through several previous transcripts or meeting summaries. A user should be able to see what is still open, what has changed, who owns each commitment, and where that information originally came from.

We also want the system to be reliable enough that users do not have to constantly double-check its work. When the system is unsure whether a statement refers to an existing commitment or whether a task's status has actually changed, it should ask for human review instead of making the update automatically. At the same time, we do not want every update to require manual review, so we will track how often human review is needed and how long those reviews take.

6. Tech Stack

Layer	Primary Technology	Key Responsibility
Mobile Client	SwiftUI	UI, Dashboards, Task Card Review, State Machine Picker

Audio Playback	AVFoundation	Sub-second audio seeking from transcript/task citations
Local OS Sync	Apple EventKit	Native Calendar matching and Apple Reminders sync
Tunneling	Cloudflare Tunnel / Tailscale	Secure remote connection from iOS to Mac backend
Backend API	FastAPI / Uvicorn	Routing, background queue, project access guards
Speech & Diarization	MLX-Whisper + pyannote.audio	Local Apple Silicon ASR and speaker separation
LLM & Extraction	Claude + Pydantic	Structured summaries, task extraction, deduplication check
Database & Search	PostgreSQL 16 + pgvector	Relational state, 5-state task history, vector RAG search
Safe Deletion	send2trash	Recoverable local file removal via macOS Trash
Evaluation	Ragas + jiwer + pyannote.metrics	WER, DER, and RAG faithfulness benchmarks

7. Data and licensing constraints

For development and evaluation, we plan to use a combination of existing meeting datasets and purpose-recorded meetings from our own project team or study groups. One dataset we are considering is the AMI Meeting Corpus, which includes recordings, transcripts, and annotations from groups working together across multiple meetings. This makes it useful for testing parts of our system that need to connect information over time, although we would not assume that performance on AMI represents performance in our intended PhD advisor and researcher setting. We may also use QMSum for additional summarization or retrieval testing, while our own meeting sequences will focus specifically on commitments, decisions, deadline changes, task updates, and other cross-meeting scenarios our system needs to handle.

For any meetings we record ourselves, everyone involved will agree to participate before recording or processing begins. Participants will know what is being collected, how we plan to use it, what information will be kept, and whether any content will be sent to an external AI provider. Participants will also be able to request that their data be excluded or deleted. We will not assume that real research lab meetings or existing CCB recordings are available to us; if we use them later, we will first confirm participant consent, access permissions, and any additional institutional requirements.

Before using any outside dataset, model, library, or CCB code, we will check and document its license and usage restrictions. Private meeting recordings, transcripts, speaker information, embeddings, and other derived data will be kept local and will not be committed to our public GitHub repository.

8. Compute budget and tooling

We expect the project to have relatively low infrastructure costs because we are not training or fine-tuning a large model. Development will use the team's existing laptops, local storage, GitHub, SQLite, and free or open-source development tools wherever possible. No additional hardware purchases are expected, and because the MVP is designed as a local-first application, we do not currently expect to need paid cloud hosting or a production database.

Our main expected cost is API/model usage for structured extraction, cross-meeting reconciliation, rubric-graded summarization, historical Q&A, and evaluation. Our evaluation plan currently targets a small number of purpose-recorded meeting sequences, with the exact count depending on whether teaching staff can point us to existing pre-recorded advisor/student sequences we could use instead (see Section 6); as a planning upper bound, we estimated cost assuming around 18 short meetings across six three-meeting sequences. Because the exact cost will depend on the model we select, transcript length, number of evaluation runs, and retries, we will first run a small pilot and measure the actual cost per meeting and evaluation run rather than assume a fixed cost in advance. We will also cache transcripts, embeddings, and other unchanged intermediate outputs so that repeated experiments do not unnecessarily repeat the same processing.

We propose an initial total project spending cap of \$75, broken down as follows:

- **Model/API usage - up to \$50:** Reserved for LLM calls used for extraction, reconciliation, summarization, retrieval-augmented Q&A, and evaluation. Working through our own pipeline at current API rates (roughly \$1-\$2 per million input tokens and \$5-\$10 per million output tokens for a capable model, or well under \$1 per million tokens for a cheaper tier), our full 6 to 10-meeting evaluation matrix comes to well under \$50, even accounting for development iteration and repeated evaluation runs
- **Hardware - \$0 expected:** We will use hardware already owned by team members. Dedicated GPU hardware is not expected to be necessary because the primary contribution of this project is the longitudinal memory and reconciliation pipeline rather than training a new model. ([Claude API Pricing 2026](#); [OpenAI API Pricing 2026](#))
- **Local storage/deployment - \$0 expected:** The MVP is designed as a local-first, single-user application using local storage and PostgreSQL, so a paid production server, hosted database, or cloud deployment is not required for the core project. Speech-to-text and diarization also run locally through Whisper and pyannote.audio, both free for research use, so we do not expect to pay for a hosted transcription API either. ([pyannote/speaker-diarization-3.1 license](#), [Hugging Face](#))
- **Contingency - up to \$25:** Reserved for unexpected costs such as limited paid transcription, temporary cloud compute, or another service that becomes necessary during integration or final evaluation. This amount will only be used if the required functionality cannot reasonably be completed using our existing hardware or free/open-source resources.

Actual API and compute costs will be recorded throughout development. If projected spending begins to approach the \$75 cap, we will prioritize the core three-meeting longitudinal evaluation, reduce unnecessary repeated experiments, use lower-cost models when they meet our quality requirements, and reuse cached intermediate results before increasing the project budget. The final report will include the actual amount spent and, where applicable, measured cost per meeting or evaluation run.

9. Risks and mitigations

Risk	Response
Incorrect speaker assignment gives a task to the wrong person	Confirm speaker mappings, preserve unknown identities, and evaluate owner attribution separately.
Similar tasks are incorrectly merged or completed	Require project/owner/evidence consistency; review ambiguity and prioritize update precision.

Risk	Response
Silence or partial progress is mistaken for completion	Require explicit supported state changes; include negative and partial-completion fixtures.
Summaries or cleanup invent meaning	Preserve raw segments, attach evidence, and manually score faithfulness.
Too little longitudinal data	Purpose-record linked scenarios and clearly separate controlled results from natural-meeting results.
Audio integration delays the central contribution	Start with timestamped transcript imports; reuse the sponsor pipeline for audio where available.

10. Ethics and safety plan

Research meetings can include private or unpublished information, so we need to be careful about what gets recorded, stored, and carried forward. There is also the risk that the system gets something wrong, such as assigning a commitment to the wrong person, treating a suggestion as an accepted task, marking something as completed without enough evidence, or connecting an update to the wrong project. Since the system keeps information across meetings, a mistake in one meeting could affect what it shows later.

For privacy, we will only use meeting data when everyone involved has agreed to participate, keep the system local-first, avoid storing information we do not need, and allow meeting data to be deleted. We also need to account for bias in the system. Differences in accents, speaking styles, audio quality, or how people phrase things could affect transcription, speaker identification, or how accurately commitments are extracted. We will look for these types of errors during evaluation and will not make claims about groups or settings that are not represented in our data. We also will not use things like how often someone speaks or how many tasks they complete as a measure of their contribution or performance.

To reduce misuse and incorrect outputs, commitments, decisions, and updates will stay connected to the meeting evidence they came from. Suggestions will stay separate until someone actually accepts them, and a task will not be marked completed or dropped just because nobody mentioned it again. If the system is unsure about an owner, task match, or status change, it will send it for human review instead of making the change automatically, and users will be able to correct mistakes. The system will also not be designed to rank researchers, create productivity scores, or monitor people without their consent. During evaluation, we will specifically test cases like wrong speaker attribution, ambiguous references, unsupported task

updates, and hallucinated claims to see where the system fails and whether these safeguards actually help.

11. Scope boundaries

In Scope

- **Native iOS application:** Serves as the primary mobile client for audio ingestion, transcript viewing, task management, and conversational queries.
- **MP3 audio file ingestion:** Allows users to import and process local or shared MP3 audio recordings directly into the pipeline.
- **Audio-to-text transcription:** Converts spoken audio into timestamped text transcripts using existing speech-recognition models.
- **Speaker segregation/diarization:** Identifies distinct speakers across the recording to attribute spoken statements to specific participants.
- **LLM Summary creation:** Synthesizes transcripts into structured, concise meeting overviews highlighting core discussions and key takeaways.
- **Speaker Task extraction (Todo list):** Automatically identifies action items assigned to specific speakers and routes them into a human-in-the-loop review workflow for approval, editing, or rejection.
- **Task lifecycle management (Open, In-Progress, Blocked, Done, Dropped):** Tracks and transitions tasks across five explicit status states to reflect real project progress across meeting sequences.
- **Duplication filtering:** Compares newly extracted action items against previously tracked open tasks to prevent redundant entries.
- **Calendar integration / updates:** Pulls meeting metadata (attendees, dates, projects) from external calendar services and syncs task deadlines back to user schedules.
- **Meeting Session Storage:** Securely persists raw audio files, transcripts, generated summaries, and structured task data for long-term access.
- **Safe file handling:** Enforces secure local storage practices, validation checks, and cleanup routines to prevent file corruption or data leaks.
- **Access policy / Authentication Guard:** Restricts access to meeting records and project data through user authentication and permission-checking rules.
- **Evaluation measurement:** Benchmarks system performance and model accuracy using reproducible evaluation datasets and metric scoring.
- **RAG System / User LLM integration:** Enables users to ask natural-language questions about past discussions and retrieve answers grounded in historical meeting context.
- **Update meeting notes:** Allows users to manually edit, append, and save revisions to generated meeting summaries.
- **Update task todos:** Enables manual editing of task attributes, including assignees, descriptions, due dates, and status values.
- **Provide citations back to the source:** Links extracted tasks, decisions, and RAG answers directly to exact transcript timestamps and audio segments for verification.
- **Summary sharing:** Formats and exports meeting notes and action items for external sharing across communication platforms or files.

- **Todo list (task) Reminders:** Triggers scheduled reminders and highlights approaching deadlines for outstanding action items.
- **Audit Trail, Versioning, and Undo:** Records a historical log of modifications to tasks and notes, allowing users to inspect changes and revert unwanted edits.
- **Provide inspection by meeting, person, and project:** Generates aggregated, filtered dashboard views organized by meeting session, attendee, or broader project workstreams.
- **Demonstrate consent gating, selective exclusion, deletion, and project-scoped retrieval:** Implements privacy mechanisms to verify participant recording consent, exclude sensitive transcript portions, purge deleted records, and isolate data retrieval strictly to the target project.

Out of Scope

- **Transcription encryption:** Implementing cryptographic encryption at rest or in transit for stored transcripts and audio files will not be built.
- **Notification system:** Operating dedicated background push notifications, SMS alerts, or external notification dispatch infrastructure is omitted.
- Train a speech-recognition model from scratch
- Generate productivity or performance scores
- Rank students or researchers
- Monitor meetings without participant consent
- Deploy a production multi-tenant service
- Treat meeting statements as independently verified proof that real-world work occurred
- Claim broad performance across languages or meeting settings beyond those evaluated

12. Timeline

Week	Focus Area	Key Deliverables & Engineering Milestones	Milestone Checkpoint
Week 1	Setup & Data Access	Clone professor repository; set up local M4 environment (mlx-whisper, pyannote.audio, PostgreSQL + pgvector); define Pydantic schemas and API contracts.	

Week 2	Base Infrastructure	Initialize SwiftUI app shell; configure Cloudflare Tunnel; define initial database migrations; verify sample audio processing.	Checkpoint 1: Data Access
Week 3	Baseline Setup	Build baseline models (zero-shot extraction, token Jaccard matching, 500-token naive RAG); write metric scripts (jiwer, pyannote.metrics, Ragas).	
Week 4	Evaluation Harness	Finalize test fixtures (annotated multi-meeting transcripts and audio clips); build automated evaluation runner (eval_runner.py) for baseline comparisons.	Checkpoint 2: Evaluation Harness
Week 5	Audio & Ingestion Sync	Implement iOS MP3 import, consent modal, and background upload; integrate AVFoundation audio-transcript timestamp scrubbing on iOS.	
Week 6	Stateful Task Review	Deploy 5-state task lifecycle (Open → Dropped); build iOS Human-in-the-Loop review card UI; connect backend task extraction with schema validation.	Checkpoint 3: Initial Working System

Week 7	Task Reconciliation	Implement semantic deduplication engine (sentence-transformers + LLM arbiter); resolve cross-meeting shorthand mentions against persistent PostgreSQL state.	
Week 8	Historical RAG & Sync	Build hybrid search (dense + BM25) over conversational turns; implement interactive audio citation chips in iOS chat; sync approved tasks to Apple Reminders via EventKit.	
Week 9	Hardening & Delivery Prep	Build Apple TestFlight release; draft step-by-step local setup guide and API specs; freeze evaluation fixtures and test runners for reviewers.	Checkpoint 4: Packaging & Documentation
Week 10	Benchmarking & Defense	Execute full evaluation suite across baseline vs. proposed systems; compile charts and error analysis into final written report; deliver live project demonstration.	Checkpoint 5: Final Report & Live Presentation

13. Roles and responsibilities for teams (Team project only)

Member	Primary responsibility	First implementation checkpoint
Will Liu	Data contracts, extraction, and reconciliation	Task schema, source references, update rules, and ambiguous-match handling
Bryan Yang	Runtime, native iOS integration, storage, and pipeline integration	Transcript-to-task vertical slice with swift corrections and timestamp inspection
Guadalupe Cantera	Evaluation, data preparation, and quality	Consent/data protocol, sequence fixtures, annotation guide, and pilot metrics

14. Data

To evaluate system capabilities against established baselines, the project leverages public benchmarks including the AMI Meeting Corpus and QMSum. The AMI Meeting Corpus provides multi-session recordings, transcripts, and annotations of groups collaborating over time, making it an effective proxy for testing how well the pipeline links evolving information and tracks cross-meeting context. In parallel, QMSum serves as an external benchmark to evaluate query-focused summarization and historical context retrieval. Prior to incorporating either corpus or any supporting open-source checkpoints, all underlying software licenses and usage restrictions will be reviewed, documented, and adhered to.

Because existing public corpora lack the specific nuances of academic supervision, the team will build a small, curated evaluation dataset comprising annotated meeting sequences modeled directly on recurring research group interactions. These workarounds will provide gold-standard annotations for critical cross-meeting follow-through behaviors of disambiguating binding student commitments from tentative professor suggestions, capturing explicit architectural decisions, logging deadline modifications, and tracking task lifecycle transitions across the five core states (Open, In-Progress, Blocked, Done, Dropped). Crucially, these fixtures will embed realistic conversational shorthand, such as referring to an earlier multi-step experiment simply as "the rerun", providing the ground truth needed to quantitatively benchmark task extraction F1, owner attribution accuracy, cross-meeting linkage F1, and citation precision against baseline methods.

For real-world testing, the team will collect purpose-recorded meeting sequences within its own project team and internal study groups under strict informed consent and privacy protocols. Prior to recording or processing any session, all participants must explicitly agree to participate after being fully briefed on what audio and textual data is collected, how it will be processed,

how long it will be retained, and whether any transcript tokens will be transmitted to external AI API providers. Participants retain full agency to selectively exclude sensitive discussion segments or request the permanent deletion of their data at any point. Furthermore, all collected audio files, raw transcripts, speaker voice profiles, and derived embeddings will remain strictly quarantined on local machines and will never be committed to a public GitHub repository.

15. Model and system

Retrieval Augmented Generation (RAG): Directly powers the historical cross-meeting Q&A system.

Tool use or agentic orchestration: Powers the structured extraction, multi-meeting task reconciliation, and summarization pipelines.

16. Application and deployment

- Lightweight User Interface
- Local Deployment Only

17. Evaluation and responsible AI

- Custom evaluation suite beyond the baseline harness.
- Robustness testing for ambiguous references, negation, partial completion, uncertain speakers, and related edge cases.
- Safety and hallucination checks, including attribution hallucination and unsupported claims.
- Latency, token usage, and cost profiling.

Bias analysis is not currently selected as a primary evaluation focus because the pilot-scale, purpose-recorded dataset is not large or diverse enough to support meaningful demographic bias claims. We will revisit this if the evaluation dataset grows substantially.

18. Weekly Checkins

Progress Made

This week, we finalized the revised project direction following feedback from our professor and TA and developed the initial project scope, architecture, evaluation plan, responsible-use plan, and team responsibilities. We also reviewed related meeting-assistant products to better define our project's contribution and began planning the evaluation harness and three-meeting test sequences.

Top Blockers and Risks

Our main open questions are whether we will be permitted to reuse the sponsor's existing meeting pipeline, whether appropriate pre-recorded advisor/student meeting sequences are available, and whether our proposed pilot-scale evaluation dataset is sufficient for the claims we want to make. We also need to make sure that audio/transcription integration does not take development time away from the project's central contribution: cross-meeting reconciliation and persistent state.

Planned Next Steps

Next, we will confirm sponsor-code and data access, finalize team ownership, create the first labeled three-meeting development sequence, implement the transcript/task data contract and independent-meeting extraction baseline, and build a reproducible offline evaluation runner. This will give us an initial end-to-end evaluation path before implementing the full reconciliation pipeline.

Remaining questions for teaching staff: Is the proposed small sequence dataset sufficient for the intended claims? Is independent per-meeting extraction the right primary baseline? Should audio processing remain required, or can the sponsor pipeline serve as an upstream component? Are pre-recorded advisor/student meeting sequences with tasks and deadlines available?

19. Pitch Deck

[Link to Pitch Deck](#)